# 14 More lessons from 14 years at Google

This time about teams, trust, and the systems around the code.

ADDY OSMANI
FEB 12, 2026

108 5 12 Share

A while back, I wrote down 21 lessons from my time at Google. The response caught me off guard because of *which* ones stuck. It wasn't tech-specific advice. It was the stuff about people, decisions, and the messy reality of building things together.

That made me realize I'd left a lot on the table. The first list skewed toward individual craft - how to write better code, how to think about your career. But some of the hardest lessons I've learned aren't about how you work. They're about how teams work: how decisions actually get made, where coordination breaks down, what separates the groups that ship from the ones that spin.

These lessons pick up where the first left off. They're less about being a better individual engineer and more about the systems around the engineering.

## 1. The best engineers pick the right problems to solve.

Every yes is an implicit no to something else.

I've watched talented engineers burn out because they said yes to everything - every bug, every feature request, every "quick favor." Their calendar filled up with other people's priorities, and their own roadmap became a graveyard of half-finished ideas.

Sometimes it's just because they truly do care so much about the product. Protect your bandwidth from "nice to have" the same way you protect production from outages. The skill is doing the right things and letting the wrong things stay undone.

The engineers who create disproportionate impact aren't necessarily faster or smarter. They're more ruthless about what deserves their attention. They've learned that the opportunity cost of working on the wrong thing is working on the wrong thing.

## 2. If you can't say what decision you're asking for, you're not ready for the meeting.

Most meetings fail not because they're unnecessary, but because they're disguised journaling. I've sat through hundreds of hours where smart people talked around a problem without ever naming what they needed. The meeting ends with vibes and no owner.

I learned to start with the ask: approve, choose, unblock, or inform.

Just those four words changed how I prepare for every meeting. If I can't pick one, I'm not ready to take anyone's time. And when I'm on the receiving end, I've started asking "what decision do you need from me?" within the first two minutes. It sounds blunt, but people are usually relieved - they often didn't realize they hadn't defined it themselves.

The hidden cost of vague meetings isn't just the hour you lose. It's the week of drift that follows while everyone waits for clarity that never came.

## 3. "We should" is not a plan. "On Tuesday, I will" is a plan.

The difference between motion and progress is specificity.

Teams drown in intentions. I've watched roadmaps fill up with "we should improve the onboarding flow" and "we should reduce latency" and "we should document the API." Months later, the same items are still there, gathering dust and guilt. You might think that's a solved problem now that we have agentic engineering, but not quite.

Convert talk into the smallest next action someone can actually do, then put a name and a date on it. Not "we should improve onboarding" but "On Tuesday, Sarah will run three user sessions and document the top friction points."

This is about respecting that humans need traction to make progress. Vague intentions create anxiety. Specific commitments create momentum. The plan doesn't have to be perfect - it just has to be concrete enough that someone can actually start.

## 4. Slow code is sometimes a symptom. Slow decisions are always a problem.

Speed is about removing the friction that makes smart people hesitate. "Bias towards action" when you can.

When a project drags, the instinct is to blame velocity: people aren't working hard enough, the codebase is messy, there aren't enough engineers. But in my experience, slow code is often a symptom. Slow decisions are the disease.

If decisions routinely take weeks or months, look deeper. Missing context means people can't evaluate tradeoffs. Unclear ownership means everyone's waiting for someone else to decide. Fear of accountability means people hedge instead of commit.

The fastest engineering team I ever worked with wasn't the one with the best programmers. It was the one where decisions happened in hours instead of weeks because the authority was clear, the context was shared, and being wrong wasn't a career risk.

## 5. Reliability is a product feature. Treat it like one.

Users don't praise reliability but they do notice its absence.

This creates a dangerous dynamic: reliability work is invisible until it fails, which means it's perpetually under-resourced compared to shiny new features.

Error budgets are one way to make the tradeoff explicit. If your service has an SLO of 99.9% uptime, you have a "budget" of 0.1% downtime to spend on innovation. Burn through it, and you focus on reliability until you've earned it back. This is a framework for having honest conversations about risk.

The teams that maintain both velocity and reliability don't do it through heroics. They do it by treating reliability as a first-class product feature with its own roadmap, its own metrics, and its own advocates.

You wouldn't ship a feature without product review. Don't ship a system without some kind of reliability discussion.

## 6. You can't "communication" your way out of a bad interface between teams.

Team interaction modes exist for a reason: collaboration (working closely together), service (clear API and SLAs), or facilitation (one team helping another build capability).

Most cross-team pain isn't about effort or good intentions. It's about unclear boundaries and messy contracts. I've watched teams "improve communication" by adding more meetings, more Slack channels, more syncs - and it doesn't make things better.

The problem isn't that people aren't talking. It's that the interface between teams is undefined. Who owns what? What's the contract? What can team A depend on team B for, and vice versa?

Choose deliberately, and you'll need fewer meetings to make things work. Try to paper over a bad interface with communication, and you'll burn out your most collaborative people while the underlying dysfunction remains.

## 7. The best escalation comes with a proposal.

"Here's the problem" is half the job. I used to think my role was to identify issues and bring them to leadership. That's necessary but insufficient.

"Here are two options, the tradeoffs, and what I recommend" is how you get unblocked and earn trust. It shows you've done the thinking. It gives decision-makers something super specific to react to instead of an open-ended problem to solve.

It makes their job easier, which makes them more likely to give you what you need.

The difference between "I need help" and "I need you to choose between A and B, and here's why I lean toward B" is the difference between being a problem-raiser and being a problem-solver.

Both identify issues. Only one earns increasing trust and autonomy.

## 8. Avoid hero culture. Build systems that don't require heroes.

The hero is burned out, undocumented, and a single point of failure.

If one person saving the day is a recurring pattern, that's a failure mode rather than a badge of honor. I've seen teams celebrate their heroes while ignoring the dysfunction that made heroism necessary.

When they leave - and they always leave eventually - the team discovers that no one else really knows how things work. The celebration of heroism masks a systemic problem: the path for "normal humans on a normal day" doesn't work.

Make the normal path the default. Document the system. Spread the knowledge. Design for the average Tuesday, not the exceptional crisis. Heroes should be unnecessary, and if they're necessary, you should be working to make them unnecessary.

## 9. Make observability part of the feature.

A feature without telemetry is a liability in disguise.

If you ship a feature without knowing how it behaves in production, you shipped uncertainty.

I've watched teams celebrate launches only to discover weeks later that their feature was silently failing for 20% of users. They had no logs, no metrics, no dashboards but a gap where understanding should be. This can cause all kinds of pain if you want to fix it, including unshipping just to properly A/B test with observability in place.

Logs, traces, dashboards, and alerts aren't "ops work." They're how you learn. They're how you know whether the thing you built actually works for real people doing real things in real conditions.

The best engineers I know treat observability as part of the definition of done. Not "I wrote the code" but "I wrote the code and I can see it working."

## 10. Small PRs are kindness. Especially if the PR is AI generated.

Small changes are easier to review, easier to reason about, and easier to revert.

I used to write large pull requests. I liked the idea of a complete feature being reviewable at once. I was optimizing for my convenience at the expense of my reviewers' sanity. Smaller PRs are often better for everyone.

They ship faster because they don't sit in a review queue while someone tries to find an hour to understand your thousand-line diff. If you want teammates to trust your pace, make your work reviewable.

The hidden benefit is that small PRs force you to think in increments. Instead of one monolithic change, you build up capability piece by piece. Each piece gets feedback. Each piece can be rolled back independently. It's slower per-PR but faster to actual production.

## 11. When you add a team, you add edges, not just nodes.

Coordination cost grows faster than headcount.

This is why "just throw more people at the problem" often fails, and why adding heads late in a project can make it later. Every new person adds communication overhead with everyone they need to coordinate with. The graph gets denser, not just larger.

I've seen managers genuinely puzzled when a team doubled in size but output barely changed. The answer is always the same: the new edges ate the new capacity. More people meant more alignment meetings, more context-sharing, more waiting for decisions that now required more stakeholders.

The solution isn't to stop hiring. It's to be intentional about reducing edges. Clear ownership. Autonomous teams with minimal dependencies. Interfaces that let people work in parallel instead of in lockstep. The best organizations aren't the ones with the most people - they're the ones with the most leverage per person.

## 12. The migration is never just a migration

Every migration is a negotiation between the system you have, the system you want, and the people who didn't ask for either.

I've seen migrations estimated at one quarter stretch to years. Not because the technical work was wrong, but because nobody accounted for the human work: convincing teams to prioritize your migration over their roadmap, supporting the long tail of edge cases nobody knew existed, and maintaining two systems in parallel while the old one refuses to die.

The technical plan is the easy part. The hard part is designing for coexistence. You will run old and new simultaneously for longer than you think. You will discover that the "legacy" system encodes decisions nobody documented and workflows nobody remembers designing but everyone depends on. You will need a adoption strategy that doesn't require every team to drop what they're doing at once.

The migrations that actually finish share three traits: a sponsor who stays engaged past the kickoff, a team that really owns the migration instead of treating it as a side quest, and a clear deprecation date that people believe is real. Without all three, you get a migration that's perpetually "almost done" - which is worse than not starting, because now you're paying the cost of two systems indefinitely.

If you're not willing to fund the finish, don't start the migration.

## 13. AI makes drafts cheap. Taste becomes expensive.

Everyone can generate code now. The barrier to producing code, content, designs - it's largely collapsing. AI will write you ten versions of anything in the time it used to take to write one.

The differentiator is choosing: what to build, what to delete, what to simplify, what not to ship, and what "good" looks like. Taste - the ability to distinguish between options and pick the right one - becomes the scarce resource.

Use AI to explore options fast, then apply judgment ruthlessly. The engineers who thrive in this environment won't be the ones who generate the most. They'll be the ones who curate the best.

Production is cheap. Editing is expensive. Selection is everything.

## 14. Trust is a latency optimization for teams.

This is the highest-leverage thing you can build. Not a system but credibility.

When people trust you, they don't need five meetings to approve a decision. They assume competence, good intent, and follow-through. Decisions that would take weeks in a low-trust environment take hours in a high-trust one.

Every time you deliver on a promise, every time you're honest about a mistake, every time you make someone else's life easier, you're depositing into an account that will pay dividends for years.

I've watched engineers with modest technical skills accomplish enormous things because everyone trusted them. I've watched brilliant engineers accomplish little because nobody would take their calls.

The code doesn't matter if you can't get anyone to ship it with you.

## A final thought

The first time around, I said these lessons come down to staying curious, staying humble, and remembering that the work is about people. I still believe that.

But if this second list has a through-line, it's something more specific: the work is about making it easier for normal people to do extraordinary things on a normal day. A career in engineering gives you plenty of time to learn these things the hard way and I've certainly learned a lot during my time at Google so far.

I hope a few of them save you a scar or two. And if they do, share what you've figured out with someone earlier in the journey.

That's how the good lessons travel.

Subscribe to Elevate
By Addy Osmani · Hundreds of paid subscribers
Addy Osmani's newsletter on elevating your effectiveness. Join his community of 600,000 readers across social media.

giaco.maggiore@gmail.c Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.

108 Likes · 12 Restacks

108 5 12 Share

### Discussion about this post

Comments Restacks

Write a comment...

**8Lee** Digital Onigiri 12 Feb
Liked by Addy Osmani

Another well-timed piece, for sure.

AI will never account fully for "taste" nor will it ever truly know priorities as those change and evolve faster than foundation models. One thing I've learned as an engineer that "caring" can be toxic and difficult to quantify but no one can argue with the measurement of (daily) use.

Engineers who use their own products are ones who understand the problem space fully and deeply. This is in very rare supply and anything that starts with "Enterprise ___fill-in-blank___" will find that most of the staff do not actually use the very product that they are building.

Care and time and attention go quickly out the door.

LIKE (4) REPLY SHARE

1 reply by Addy Osmani

**Ivy Chen** wonderstruck 12 Feb
Liked by Addy Osmani

I really enjoyed this article! Which is a good reminder of what I want to look for in my next job.

LIKE (2) REPLY SHARE

3 more comments...

Top Latest Discussions

**The 70% problem: Hard truths about AI-assisted coding**
A field guide and why we need to rethink our expectations
DEC 4, 2024 · ADDY OSMANI

**The Prompt Engineering Playbook for Programmers**
Turn AI coding assistants into more reliable development partners
MAY 27, 2025 · ADDY OSMANI

**How to write a good spec for AI agents**
How to structure, plan, and iterate for high-performance coding agents
JAN 19 · ADDY OSMANI

See all

Ready for more?

giaco.maggiore@gmail.com Subscribe

© 2026 Addy Osmani · Privacy ∙ Terms ∙ Collection notice

Start your Substack Get the app

Substack is the home for great culture